Deep Dive into Java Memory Management: Heap, Stack, Metaspace, and Garbage Collection

Mohamed Eid

Follow

5 min read

.

Oct 31, 2024

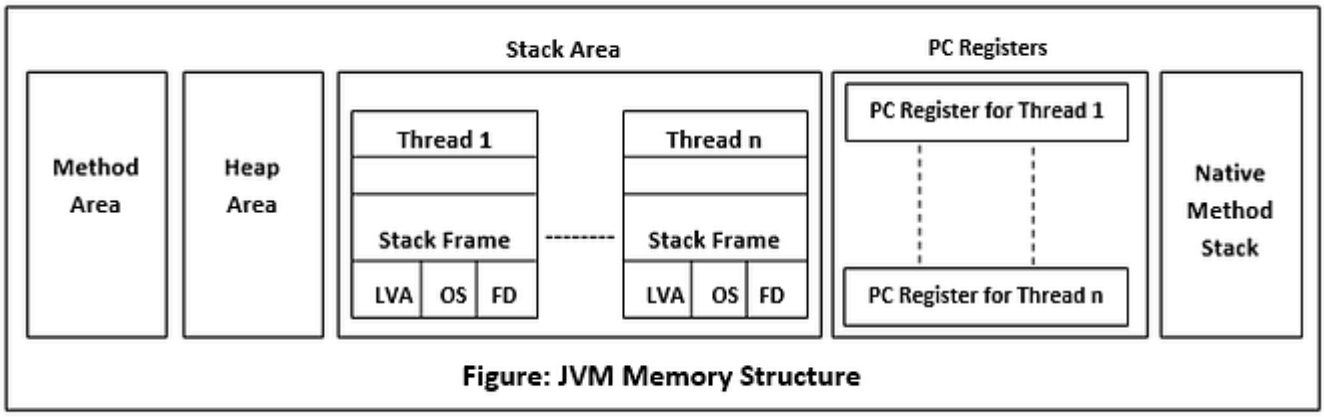
120

2

Java’s memory management model is a critical part of the Java Virtual Machine (JVM) and can make or break application performance. From managing dynamic allocation on the heap to organizing method calls in the stack, and tracking metadata in the Metaspace, each area has a distinct role in

ensuring efficient memory usage. This guide will take a deep dive into these areas, with examples and notes to clarify concepts.

Press enter or click to view image in full size



Overview of Java Memory Areas

Java's memory model divides memory into several key areas:

- **Heap**: The primary area for dynamic memory allocation, storing objects and instance variables.
- **Stack**: Manages method execution, including method call frames and local variables.
- **Metaspace**: Stores metadata for class structures, static data, and constants (replacing PermGen in Java 8).

Let’s explore each of these areas in detail.

1. Stack

The **Stack** is crucial for managing **method execution**, storing method call frames, local variables, and references to objects in the heap. Each thread has its own stack, providing thread isolation and supporting concurrent execution.

Note: You can expand your knowledge about Java reference type from [this article](#).

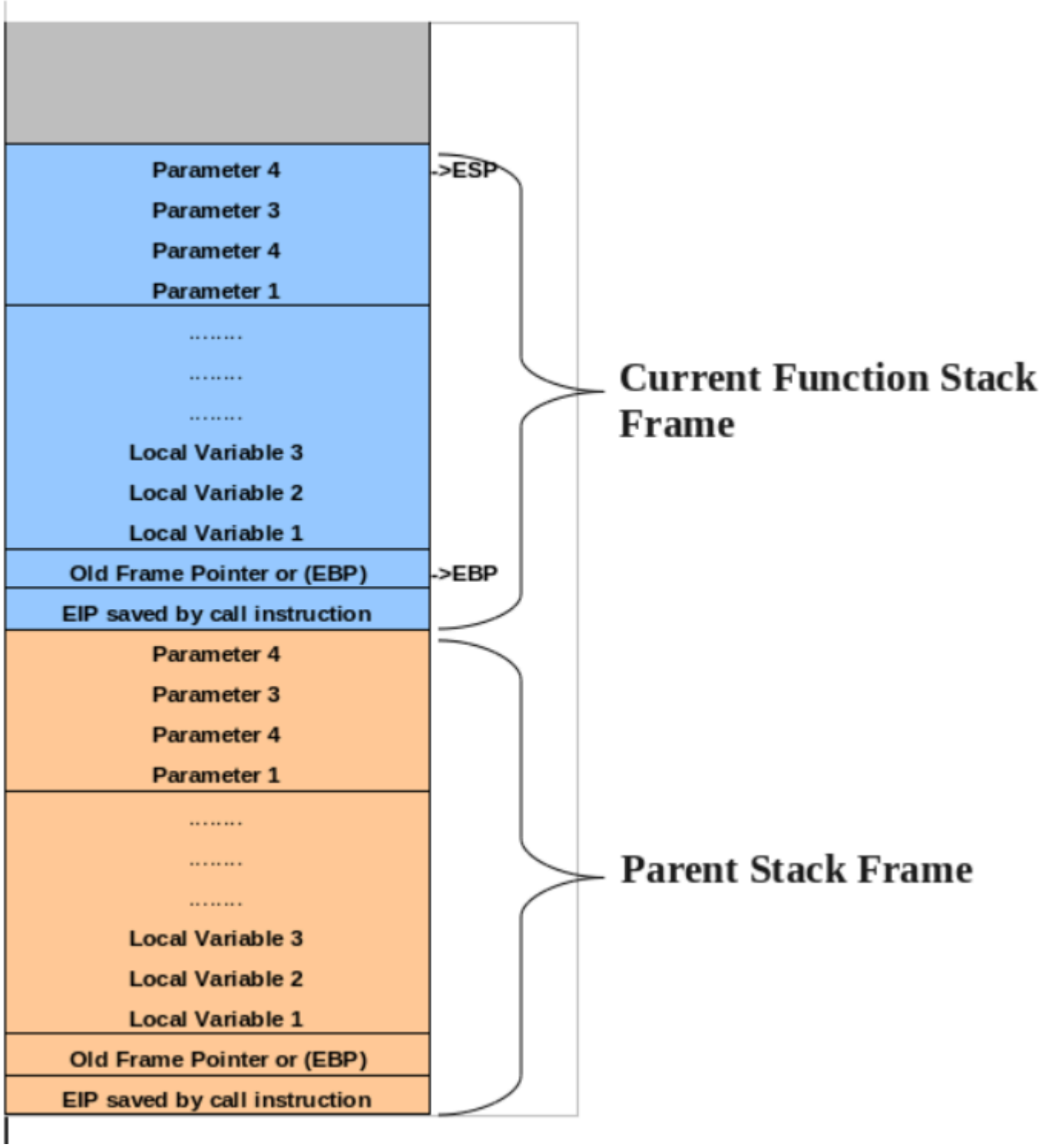
Characteristics of the Stack

- **Local Variables and References**: Local variables are stored in the stack. Here’s an example:

```
public class Main {

    public void methodA() {
```

```
        int x = 5; // Stored in the stack

        Car myCar = new Car(); // Reference is in the stack; object is in the heap

    }

}
```

. LIFO Memory Management: The stack follows a Last-In-First-Out (LIFO) approach. Each method call adds a stack frame, and the frame is removed when the method completes.



Important Notes:

- **StackOverflowError**: If the stack exceeds its allocated space (e.g., due to infinite recursion), a `StackOverflowError` occurs (ex: infinite recursion).

- **Performance**: Stack access is faster than heap access due to its simple structure, requiring no garbage collection.

2. Heap

The **Heap** is the main memory pool for dynamic allocation in Java, storing all objects created during runtime. Java's automatic garbage collection handles memory cleanup when objects are no longer referenced, making the heap essential for long-term storage.

Characteristics of the Heap:

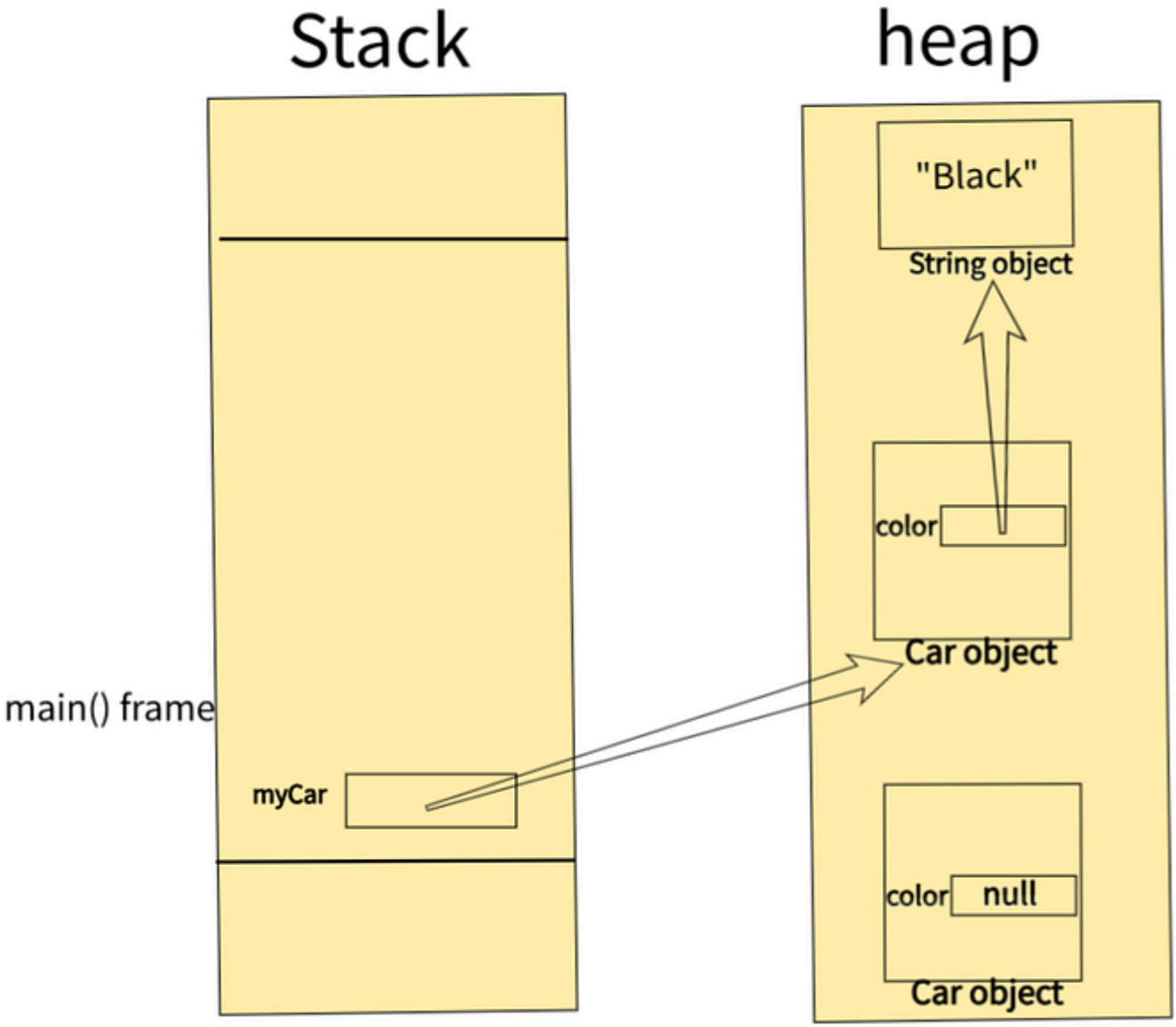
- **Instance Variables and Object Allocation**: When you create an object using `new`, it's allocated on the heap. For example:

```
class Car {

   public String color;

}
```

```
public class Main {
```

```
    public static void main(String[] args) {


        new Car();          // Anew Car object Allocated on the heap, but its reference is not preserved! (not accesible/referenced) Garbage collection eligible):


        Car myCar;          // Reference type Varaiable (allocated on the stack) (pointing to null)


        myCar = new Car(); // New Car object Allocated on the heap and its reference stored on myCar on the stack.


        mycar.color = "Black"; // New String object allocated in heap and its reference is stored in instance reference variabl color of the object


    }


}
```

Press enter or click to view image in full size



- **Garbage Collection**: Once objects lose references (e.g., after a variable goes out of scope), they become eligible for garbage collection.

Important Notes:

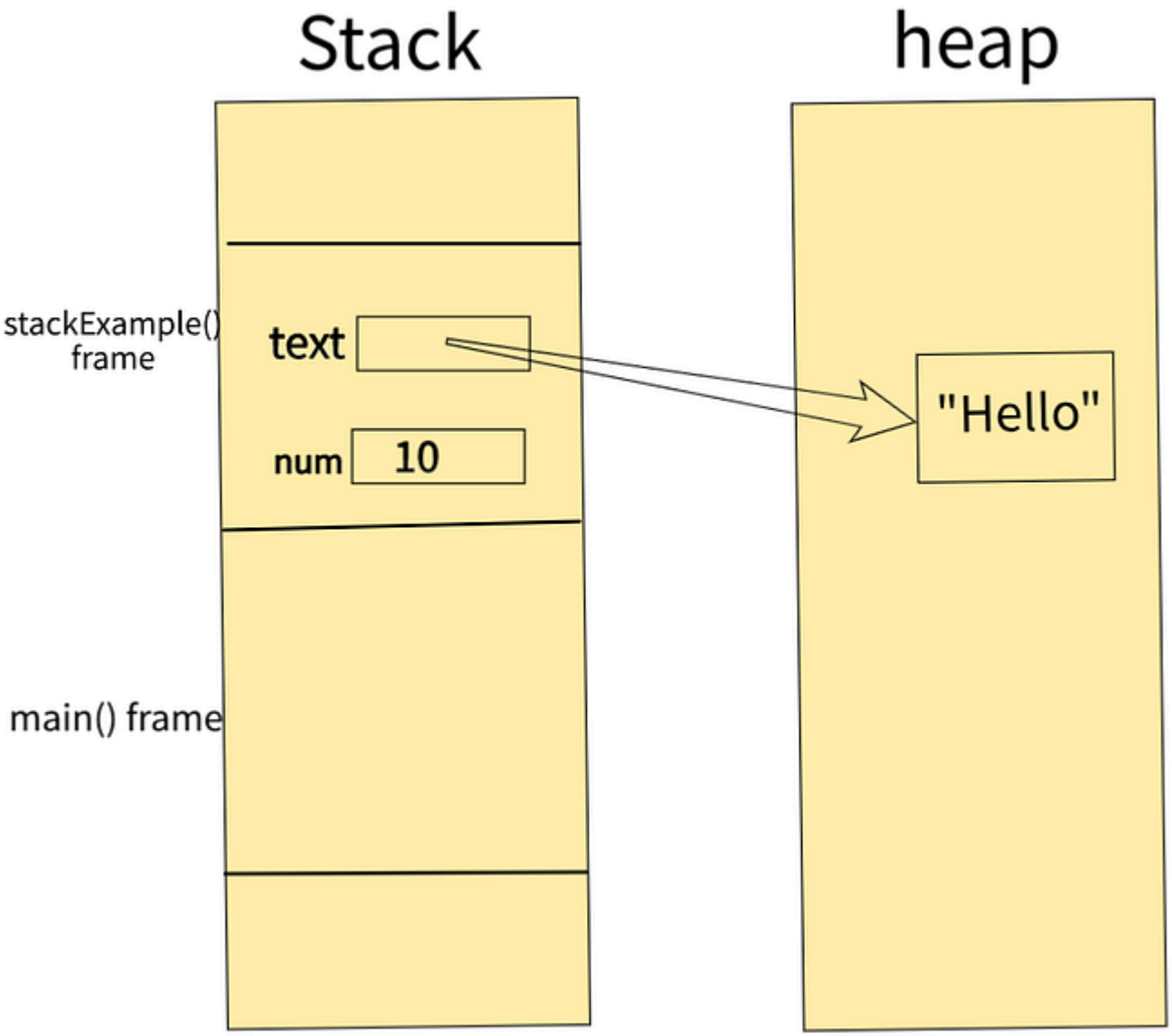
- **OutOfMemoryError**: When the heap is full and cannot allocate new memory, Java throws an `OutOfMemoryError`.

- **Access**: Heap is Shared across all threads and accessible to every method call frame.

Example: Stack vs. Heap Storage

```
public class Example {

   public void stackExample() {

       int num = 10; // Stored in the stack

       String text = "Hello"; // Reference is in the stack, but "Hello" (object of class String) is in Heap (in the string pool area)

   }

}
```

Press enter or click to view image in full size



3. Metaspace (Method Area / Class Area)

The **Metaspace** stores metadata about loaded classes and methods, such as field information, method names, and constants. Metaspace offers flexible memory allocation for class metadata.

Characteristics of Metaspace

- **Class Metadata**: Metaspace stores class structures, including field and method information.
- **Constant Pool**: Stores constant values like method names and literals.
- **Static Variables**: Houses class-level static variables and constant data. For example:

```
class Example {

   static int count = 0; // Stored in Metaspace

}
```

Garbage Collection of Metaspace

Unused classes can be unloaded and their memory reclaimed by the JVM when they are no longer referenced. This is especially important for applications that load and unload many classes dynamically.

Garbage Collection (GC)

Garbage Collection is an automatic memory management process that frees memory occupied by unreferenced objects. In Java, GC operates through a combination of minor and major collection cycles.

Phases of Garbage Collection

1. **Marking**: Identifies all live objects that are still referenced.
2. **Copying/Compacting**: Moves live objects to reduce fragmentation and organize memory efficiently.
3. **Sweeping**: Reclaims memory used by dead objects.

Example: Manual GC Hint

While Java GC is automatic, you can **request** a garbage collection cycle using `System.gc()`, though this is just a suggestion to the JVM.

```
public class Main {

    public static void main(String[] args) {

        Car myCar = new Car();

        myCar = null; // Eligible for GC

        System.gc();  // Request GC

    }

}
```

Note: `System.gc()` does not guarantee immediate garbage collection. It merely suggests it to the JVM.

Garbage Collection

Garbage collection is the process of freeing space in the heap or the nursery for allocation of new objects. This section describes the garbage collection in the JRockit JVM.

The Mark and Sweep Model

The JRockit JVM uses the *mark and sweep* garbage collection model for performing garbage collections of the whole heap. A mark and sweep garbage collection consists of two phases, the *mark phase* and the *sweep phase*.

During the mark phase all objects that are reachable from Java threads, native handles and other root sources are *marked* as alive, as well as the objects that are reachable from these objects and so forth. This process identifies and marks all objects that are still used, and the rest can be considered garbage.

During the sweep phase the heap is traversed to find the gaps between the live objects. These gaps are recorded in a *free list* and are made available for new object allocation.

The JRockit JVM uses two improved versions of the mark and sweep model. One is *mostly concurrent mark and sweep* and the other is *parallel mark and sweep*. You can also mix the two strategies, running for example mostly concurrent mark and parallel sweep.

Mostly Concurrent Mark and Sweep

The *mostly concurrent mark and sweep strategy* (often simply called *concurrent garbage collection*) allows the Java threads to continue running during large portions of the garbage collection. The threads must however be stopped a few times for synchronization.

The mostly concurrent mark phase is divided into four parts:

- *Initial marking*, where the root set of live objects is identified. This is done while the Java threads are paused.
- *Concurrent marking*, where the references from the root set are followed in order to find and mark the rest of the live objects in the heap. This is done while the Java threads are running.
- *Precleaning*, where changes in the heap during the concurrent mark phase are identified and any additional live objects are found and marked. This is done while the Java threads are running.
- *Final marking*, where changes during the precleaning phase are identified and any additional live objects are found and marked. This is done while the Java threads are paused.

The mostly concurrent sweep phase consists of four parts:

- Sweeping of one half of the heap. This is done while the Java threads are running and are allowed to allocate objects in the part of the heap that isn’t currently being swept.
- A short pause to switch halves.

- Sweeping of the other half of the heap. This is done while the Java threads are running and are allowed to allocate objects in the part of the heap that was swept first.
- A short pause for synchronization and recording statistics.

Parallel Mark and Sweep

The parallel mark and sweep strategy (also called the *parallel garbage collector*) uses all available CPUs in the system for performing the garbage collection as fast as possible. All Java threads are paused during the entire parallel garbage collection.

Generational Garbage Collection

The nursery, when it exists, is garbage collected with a special garbage collection called a *young collection*. A garbage collection strategy which uses a nursery is called a g*enerational garbage collection strategy*, or simply *generational garbage collection*.

The young collector used in the JRockit JVM identifies and promotes all live objects in the nursery that are outside the keep area to the old space. This work is done in parallel using all available CPUs. The Java threads are paused during the entire young collection.

Dynamic and Static Garbage Collection Modes

By default, the JRockit JVM uses a dynamic garbage collection mode that automatically selects a garbage collection strategy to use, aiming at optimizing the application throughput. You can also choose between two other dynamic garbage collection modes or select the garbage collection strategy statically. The following dynamic modes are available:

- `throughput`, which optimizes the garbage collector for maximum application throughput. This is the default mode.
- `pausetime`, which optimizes the garbage collector for short and even pause times.
- `deterministic`, which optimizes the garbage collector for very short and deterministic pause times. This mode is only available as a part of Oracle JRockit Real Time.

The major static strategies are:

- `singlepar`, which is a single-generational parallel garbage collector (same as `parallel`)
- `genpar`, which is a two-generational parallel garbage collector
- `singlecon`, which is a single-generational mostly concurrent garbage collector
- `gencon`, which is a two-generational mostly concurrent garbage collector

For more information on how to select the best mode or strategy for your application, see Selecting and Tuning a Garbage Collector.

Compaction

Objects that are allocated next to each other will not necessarily become unreachable ("die") at the same time. This means that the heap may become fragmented after a garbage collection, so that the free spaces in the heap are many but small, making allocation of large objects hard or even impossible. Free spaces that are smaller than the minimum thread local area (TLA) size can not be used at all, and the garbage collector discards them as *dark matter* until a future garbage collection frees enough space next to them to create a space large enough for a TLA.

To reduce fragmentation, the JRockit JVM compacts a part of the heap at every garbage collection (old collection). Compaction moves objects closer together and further down in the heap, thus

creating larger free areas near the top of the heap. The size and position of the compaction area as well as the compaction method is selected by advanced heuristics, depending on the garbage collection mode used.

Compaction is performed at the beginning of or during the sweep phase and while all Java threads are paused.

For information on how to tune compaction, see Tuning the Compaction of Memory.

External and Internal Compaction

The JRockit JVM uses two compaction methods called *external compaction* and *internal compaction*. External compaction moves (evacuates) the objects within the compaction area to free positions outside the compaction area and as far down in the heap as possible. Internal compaction moves the objects within the compaction area as far down in the compaction area as possible, thus moving them closer together.

The JVM selects a compaction method depending on the current garbage collection mode and the position of the compaction area. External compaction is typically used near the top of the heap, while internal compaction is used near the bottom where the density of objects is higher.

Sliding Window Schemes

The position of the compaction area changes at each garbage collection, using one or two sliding windows to determine the next position. Each sliding window moves a notch up or down in the heap at each garbage collection, until it reaches the other end of the heap or meets a sliding window that moves in the opposite direction, and starts over again. Thus the whole heap is eventually traversed by compaction over and over again.

Compaction Area Sizing

The size of the compaction area depends on the garbage collection mode used. In throughput mode the compaction area size is static, while all other modes, including the static mode, adjust the compaction area size depending on the compaction area position, aiming at keeping the compaction times equal throughout the run. The compaction time depends on the number of objects moved and the number of references to these objects. Thus the compaction area will be smaller in parts of the heap where the object density is high or where the amount of references to the objects within the area is high. Typically the object density is higher near the bottom of the heap than at the top of the heap, except at the very top where the latest allocated objects are found. Thus the compaction areas are usually smaller near the bottom of the heap than in the top half of the heap.

Comparing Heap, Stack, and Metaspace

Here's a comparison to summarize the roles of each memory area:

Press enter or click to view image in full size

Feature	Heap	Stack	Metaspace
Purpose	Stores objects, instance variables	Holds method calls, local variables	Stores class metadata and static data
Managed by	JVM Garbage Collector	JVM, automatically	JVM, partially managed by GC
Scope	Global (within JVM process)	Limited to method/block scope	Global
Lifetime	Until no references remain	Until method completes	Until class unloading
Speed	Slower	Faster	Moderate
Error	OutOfMemoryError	StackOverflowError	OutOfMemoryError
Accessibility	Shared across threads	Each thread has its own stack	Shared across threads

Conclusion

Java memory management is a powerful, automated system that optimizes memory use by dividing memory into specialized areas, each tailored for specific tasks. Understanding the heap, stack, and metaspace, along with the workings of garbage collection, is essential for building efficient and performant Java applications. By using these areas effectively, Java ensures robust and scalable memory management, making it one of the most dependable programming languages for large-scale applications.